

CS-112

Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

COURSE INSTRUCTOR : MS. HABIBA ARSHAD

Today we will talk about!

- Parameters in Java
- Anonymous Objects
- This Keyword
- Static Members

Arguments Passing in Java

- The two most prevalent modes of passing arguments to methods are “passing-by-value” and “passing-by-reference”.
- Different programming languages use these concepts in different ways. **As far as Java is concerned, everything is strictly *Pass-by-Value*.**

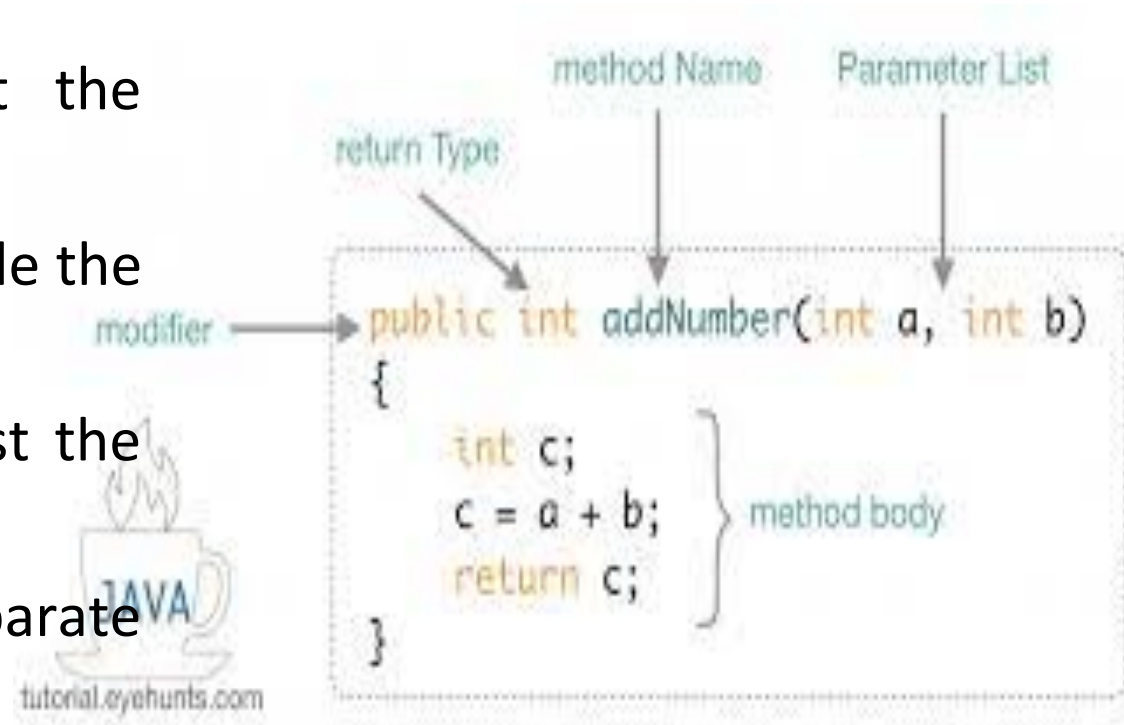
Pass-by-Value

1. When a parameter is pass-by-value, the caller and the called method operate on two different variables which are copies of each other.
2. Any changes to one variable don't modify the other.
3. It means that while calling a method, parameters passed to the called method will be clones of original parameters.
4. Any modification done in called method will have no effect on the original parameters in caller method.

Parameters in Java

Declaring parameters

- A method that accepts parameters must list the parameters in the method declaration.
- The parameters are placed in a *parameter list* inside the parentheses that follow the method name.
- For each parameter used by the method, you list the parameter type followed by the parameter name.
- If you need more than one parameter, you separate the parameters with commas.



Parameter Passing in Java

1. The fundamental concepts in any programming language are “values” and “references”.
2. In Java, Primitive variables store the actual values, whereas Non-Primitives store the reference variables which point to the addresses of the objects they're referring to.
3. Both values and references are stored in the stack memory.

Passing Arguments to Functions

// Class to test Dog objects

```
public class DogTestClass {  
    public static void main(String[]  
args) {  
        Dog one = new Dog();  
        one.bark(70); // Woof! Woof!  
        Dog two = new Dog();  
        two.bark(8); // Yip! Yip!  
        Dog three = new Dog();  
        three.bark(35); // Ruff! Ruff!  
    }  
}
```

```
class Dog {  
    int size;  
    String name;  
    void bark(int size) {  
        if (size > 60) {  
            System.out.println("Woof! Woof!");  
        } else if (size > 14) {  
            System.out.println("Ruff! Ruff!");  
        } else {  
            System.out.println("Yip! Yip!");  
        }  
    }  
}
```

Passing more than one Arguments to function

```
public class TestStuff
{
    public static void
    main(String args[])
    {
        Test t=new Test();
        int n1=12, n2=34;
        t.takeTwo(12,34);
        t.takeTwo(n1,n2);
    }
}
```

```
class Test
{
    void takeTwo(int x, int y)
    {
        int sum=x+y;
        System.out.print("sum =
"+ sum);
    }
}
```

```
run:
sum = 46
sum = 46
BUILD SUCCESSFUL (total time: 0 seconds)
```

Anonymous Object

- An object which has no reference variable is called **anonymous object in Java**.
- Anonymous means nameless.
- If you **want to create only one object in a class** then the anonymous object is a good approach.

Anonymous Object

-Declaration of Anonymous object

The syntax to create an anonymous object in Java is given below.

Syntax:

For example:

```
new Student();
```

We can pass the parameter to the constructor like this:

```
new Student("Shubh");
```

If you want to call a method through the anonymous object then you can write code like this:

```
new Student().display();
```

We can also pass the parameter to the calling method like this:

```
new Student().display("Shubh", 25);
```

This keyword

this keyword in Java is a reference variable that refers to the current object of a method or a constructor.

The main purpose of using this keyword in Java is to remove the confusion between class attributes and parameters that have same names.

This keyword

Following are various uses of 'this' keyword in Java:

It can be used to refer instance variable of current class

It can be used to invoke or initiate current class constructor

It can be passed as an argument in the method call

It can be passed as argument in the constructor call

It can be used to return the current class instance

Example: refer instance variable of current class

```
class Account{
int a;
int b;

public void setData(int a ,int b){
    a = a;
    b = b;
}
public void showData(){
    System.out.println("Value of A =" +a);
    System.out.println("Value of B =" +b);
}
public static void main(String args[]){
    Account obj = new Account();
    obj.setData(2,3);
    obj.showData();
}
}
```

Value of a & b is shown as zero? To correct the error append line # 6 & 7 with "this" keyword.

```
this.a =a;
this.b =b;
```

Static Methods In Java

- ❑ In Java, static members are those which belongs to the class and you can access these members without instantiating the class.
- ❑ The static keyword can be used with methods, fields, classes (inner/nested), blocks.

Static Methods –

- ❑ You can create a static method by using the keyword *static*. **Static methods can access only static fields, methods.**
- ❑ To access static methods there is no need to instantiate the class, you can do it just using the class name as

```
public class MyClass {  
    public static void sample(){  
        System.out.println("Hello");  
    }  
    public static void main(String args[]){  
        MyClass.sample();  
    }  
}
```

Output

Hello

Static Members In Java

Static Blocks –

- ❑ These are a block of codes with a static keyword.
- ❑ In general, these are used to initialize the static members.
- ❑ JVM executes static blocks before the main method at the time of class loading.

Example

```
public class MyClass {  
    static{  
        System.out.println("Hello this is a static block");  
    }  
    public static void main(String args[]){  
        System.out.println("This is main method");  
    }  
}
```

Output

```
Hello this is a static block  
This is main method
```

Example

```
class MathUtils {  
    // Static method  
    static int square(int num) {  
        return num * num;  
    }  
  
    public static void main(String[] args) {  
        // Calling the static method without creating  
        an object  
  
        int result = MathUtils.square(5);  
  
        System.out.println("Square of 5: " + result);  
    }  
}
```

Example

```
class Student {  
    String studentName; // Instance variable (Non-static)  
    static String uniName = "University of Wah"; // Static variable  
    // Static method trying to access non-static variable gives error  
    static void showDetails() {  
        System.out.println("Student Name: " + studentName);  
        System.out.println("School Name: " + uniName);  
    }  
  
    public static void main(String[] args) {  
        // Calling static method without creating an object  
        Student.showDetails();  
    }  
}
```

Example (Correct way to access non static members) Method 1

```
class Student {  
    String studentName; // Instance variable  
    static String uniName = " University of Wah "; // Static variable  
    // Constructor  
    Student(String name) {  
        this.studentName = name;  
    }  
    // Non-static method (Correct way)  
    void showDetails() {  
        System.out.println("Student Name: " + studentName);  
        System.out.println("School Name: " + uniName);    }  
    public static void main(String[] args) {  
        Student s1 = new Student("Ahmed");  
        s1.showDetails(); // Calling non-static method using an object  
    }  
}
```

Example (Correct way to access non static members in static methods) Method 2

```
class Student {  
    String studentName; // Instance variable  
    static String uniName = " University of Wah "; // Static variable  
    // Constructor  
    Student(String studentName) {  
        this.studentName = studentName;  
    }  
    static void showDetails() {  
        Student s1 = new Student("Ahmed"); // Creating an object  
        System.out.println("Student Name: " + s1.studentName);  
        System.out.println("School Name: " + uniName); }  
    public static void main(String[] args) {  
        showDetails(); // Calling static method  
    }  
}
```